SKYNYC · OPERATOR GUIDE 4

Replay & Detector Tuning

Re-running the event detectors over retained history — the procedure, the mental model that makes it work, and the discipline that makes it worth doing.

Covers: the offset model · the replay procedure · the tuning loop · recovery cases | August 2026

---

1. Why replay exists

The aircraft API serves at most one hour of the past. Kafka's seven-day retained log is therefore the only history this system has — and replaying it is the entire mechanism by which detector thresholds get tuned. Change a threshold, re-run the detectors over the same week of real traffic, and compare the precision/recall score before and after. Without replay you would be tuning against whatever happens to fly by today; with it, every candidate threshold is evaluated against the identical, already-scored history.

2. The one mental model you must hold

Stream processing stores its position in each query's **checkpoint directory** — not in a Kafka consumer group. Each query uses a private, auto-generated group id and never commits offsets back to Kafka. Three consequences you will actually feel:

1. Consumer-group tooling will never show the stream's position or lag. Judge progress from database freshness, never from group listings.
2. A starting-position option is honored *only when a query starts with a fresh checkpoint*. Setting it on an existing checkpoint does nothing, silently.
3. Therefore replay = **delete that query's checkpoint AND set the starting position, together**. Either one alone quietly no-ops — the most common replay mistake there is.

3. The replay procedure

All on the server, from the project directory. Steps in this order — the order is the procedure:

1. **Back up first.**
```
make backup     # database dump to the cloud backups container
```
2. **Choose the window** — an explicit UTC start time, converted to epoch milliseconds. Confirm it is inside the topic's 7-day retention; outside it, the data no longer exists and no procedure recovers it.
3. **Stop the stream.**
```
docker compose stop spark
```
4. **Delete the rows the replay will re-emit** — scoped to the window, nothing more:
```
make psql
DELETE FROM flight_events WHERE event_ts >= '<window start, UTC>';
```
5. **Delete only the events checkpoint.** The bronze and live checkpoints stay — they are separate queries with separate positions.
```
rm -rf checkpoints/events
```
6. **Set the start position** in `.env`:

```
EVENTS_REPLAY_FROM=<epoch milliseconds>
```

7. **Start and watch.**
   ```
   docker compose up -d spark
   docker compose logs -f spark | grep "events batch"
   ```
   Replaying a few days of history takes minutes; batches are size-bounded so the box stays responsive.
8. **When caught up, unset** `EVENTS_REPLAY_FROM`. It is ignored once the new checkpoint exists (consequence #2 above), so leaving it is harmless — but misleading to the next reader.
9. **Score.** Rebuild the marts and read the quality numbers:
   ```
   make dbt-build
   make psql
   SELECT * FROM analytics.mart_detection_quality ORDER BY quality_date, airport;
   ```

Deterministic event identities make overlapping windows safe — a replayed event lands on the same row it produced last time. That is the safety net, not a license to skip step 4.

---

4. The tuning discipline

Thresholds live as named constants at the top of each detector module. The loop that improves them without fooling you:

1. Record the current score for the window you are about to tune against.
2. Change **one** threshold. Two changes produce a score delta nobody can attribute, and the next session starts from zero knowledge.
3. Run the replay procedure above over the same window.
4. Record: threshold, old value, new value, precision before/after, recall before/after.
5. Keep or revert. Then the next threshold.

> **WATCH BOTH NUMBERS**
> A recall improvement that costs precision is not an improvement; it is a different trade. The targets are 85% precision and 80% recall together — a detector that fires on everything hits perfect recall and is worthless. Never ship on one number.

Real captures beat synthetic ones: when a tuning session surfaces an interesting track — a missed arrival, a false holding — capture it into the regression suite while the log still has it:

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic adsb.states.raw --from-beginning \
  --timeout-ms 30000 2>/dev/null \
  | python3 scripts/capture_sequence.py --icao24 <aircraft> --out tests/fixtures/sequences/<case>.json
```

5. Recovery cases that use the same machinery

SITUATION	WHAT TO DO	WHY IT IS SAFE
Events checkpoint corrupted after a hard crash	The failing query names itself in the log; run the full procedure with the window set to where events stop looking healthy	Idempotent event identities dedupe the overlap
Bronze or live checkpoint corrupted	Deleting either is safe on its own: bronze re-reads the retained log, live rebuilds within one trigger	Bronze dedupes within its watermark; live is an idempotent upsert of current positions
Events table polluted by a bad detector build	Fix the code first, then the full procedure over the affected window	Delete-then-replay order matters — replaying first re-pollutes
Gap in the raw feed itself	Nothing. Gaps are legitimate data; detectors handle them via timeouts	The upstream cannot backfill; absence is the honest record

6. What does not work — do not rediscover these

ATTEMPT	RESULT
Resetting a consumer group's offsets	Zero effect — the stream does not use one
Setting the start position without deleting the checkpoint	Silently ignored
Deleting the checkpoint without a start position	Replays from the earliest retained offset — usually far more than you wanted
Restarting the service to "clear state"	It resumes exactly where it left off; that is the feature
Tuning two thresholds at once	An unattributable score change and a wasted window